DACB 2.0.0

Deterministic Affine Chain Benchmark

Canonical C11 • Independent Python twin • Exact byte-identity gates

C11 | Python twin | 14-stage gate | MIT

Technical overview & verification evidence

Canonical C11 · independent Python digital twin · exact report identity · bounded exhaustive verification

PORTABLE C11 | PYTHON TWIN | 14-STAGE GATE | MIT

What DACB verifies

A narrow correctness framework for long deterministic dependency chains.

The question

Can an implementation preserve the program, execute the complete chain, respect exact arithmetic bounds, agree with independent mathematical oracles, detect semantic drift, and reject a confidently repeated wrong answer?

DACB treats agreement as evidence only when the paths are independently computed. Two matching answers are not automatically proof.

Canonical recurrence

$$x[n+1] = (1664525 \cdot x[n] + 1013904223) \bmod 2^{32}$$

$$x[0] = 1$$

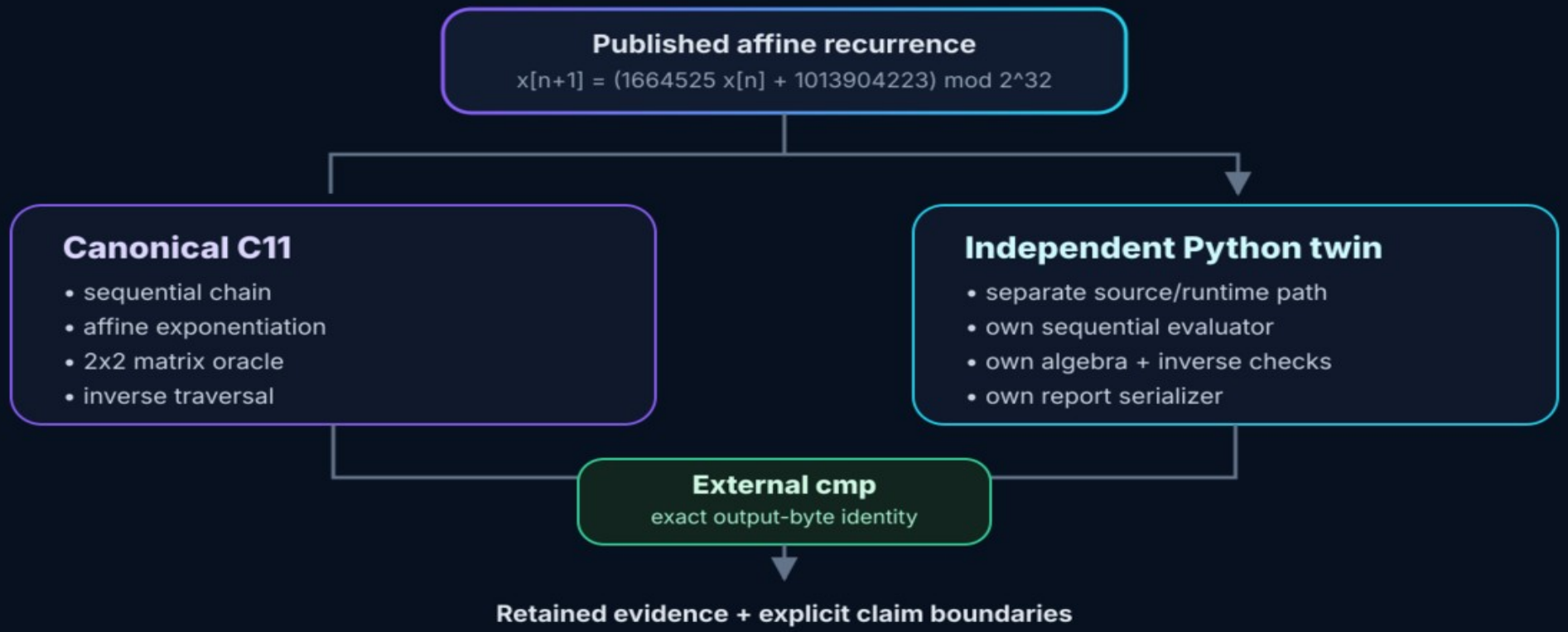
Simple enough for exact algebraic oracles; deep enough to exercise long dependency chains.

Independent paths, external comparison

The specification is shared. Runtime state and implementation code are not.

Verification architecture

Shared specification, independent implementation paths, external comparison.



Four exact state checks

Each check attacks the same recurrence from a different computational path.

01

Sequential chain

Directly executes every transition.

02

Affine exponentiation

Binary exponentiation of an affine transform.

03

2×2 matrix oracle

Homogeneous matrix exponentiation.

04

Inverse traversal

Rewinds n steps and must recover the seed.

Forward checks must agree on final state

Reverse traversal must reconstruct x[0]

Unsigned arithmetic and explicit bounds keep the C11 semantics defined; the Python twin independently reproduces the published mathematics.

Retained release evidence

Results recorded by the DACB 2.0.0 release and reproduced by the included verifier.

5,596,492
cross-language seed/step cases

22,377,804
extended C seed/step cases

1,000,000
dependent transitions in stress gate

20×
repeat report identity

Canonical 5,000-step vector

last	4125282121
sum_mod	585303980
weighted_mod	554861868

Canonical report SHA-256

```
1d2ffc1bd5703d8514a62226e42bc007
798f454c9c935d974071d6add57d7679
```

Identical bytes: GCC C11 · Clang C11 · Python twin

Release gate

The full verifier is executable, fail-closed and retained in the repository.



DACB_RELEASE_VERIFICATION_PASS

```
Run: bash scripts/verify_release.sh
```

Evidence boundary

The project is explicit about what the verification does - and does not - establish.

Supported by the release gate

- ✓ Deterministic C11 recurrence under unsigned semantics
- ✓ Known vectors reproduce
- ✓ Four state checks agree on tested workloads
- ✓ Documented reduced domains are exhaustive
- ✓ GCC / Clang / Python canonical bytes agree
- ✓ Listed adversarial mutations are detected or rejected

Not claimed

- — Full $2^{32} \times 2^{32}$ seed/step enumeration
- — Cryptographic security or randomness quality
- — Impossible checksum collisions
- — Byte-identical machine code
- — Deterministic wall-clock timing across machines
- — Physical phase-transition meaning for the lattice fixture

Runnable repository

Source-first layout with retained evidence and reproducibility controls.

`canonical_c11/`	canonical library + CLI
`python_twin/`	independent twin
`tests/`	C, Python + adversarial fixtures
`scripts/`	release verifier + benchmark utility
`docs/`	math, architecture + boundaries
`evidence/`	sealed verification records
`provenance/`	origin archive + original manifest record

Quick start

```
make
make test
make cross
bash
scripts/verify_release.sh
sha256sum -c
MANIFEST.sha256
```

Offline · no credentials · no elevated privileges

DACB 2.0.0

Deterministic claims should stay as inspectable as the code that produces them.